AgentHAB: Automating OpenHAB Rule Generation with Multi-Agent Policy and Validation

Roxie Reginold
Department of Computer Science
Toronto Metropolitan University
Toronto, Canada
roxie.reginold@torontomu.ca

Manar H. Alalfi
Department of Computer Science
Toronto Metropolitan University
Toronto, Canada
manar.alalfi@torontomu.ca

***Abstract*—The complexity of IoT device configuration can compromise security through misconfiguration and cross-app interface threats. Previous efforts have utilized Large Language Models (LLMs) for natural language policy translation, but often lack automated mechanisms for systematic conflict resolution. We propose to fill this gap with a multi-agent, conflict-aware framework for the OpenHAB platform. This research preview introduces AgentHAB, which reengineers the authoring process. It consists of a Policy Generation Agent with a Validator Agent. The generator uses retrieval-augmented generation (RAG) from a curated corpus of OpenHAB grammar to draft rules. The validator agent then iteratively checks these rules against formal syntax and static constraints, forcing the generator to self-repair. Our initial research concludes: (1) the validation and refinement loop significantly improves syntactic validity over a single-shot baseline; (2) the pipeline is robust to paraphrased user requests; and (3) context retrieval is critical for correct generation. This work presents a "proof-of-concept" and a clear research plan for achieving safe, context-aware rule generation for smart home ecosystems. This work aims to spark discussion on reliable agentic reengineering for IoT policy validation.**

***Index Terms*—IoT Security, OpenHAB, Smart Home Generation, Policy Generation, Agentic AI, Automation**

I. Introduction

The rapid development and deployment of IoT devices in smart homes and public spaces involves automation such as smart locks, lights and thermostats. However, the complexity of device configuration and writing policies can compromise security through misconfiguration, as most users lack that deep technical knowledge. Research has shown that secure unrelated IoT devices can cause interactions that lead to cross-app interface threats [1].

Previous approaches remain limited by reliance on manual intervention and a lack of automated, systematic conflict resolution. Furthermore, the rapid evolution of LLMs and the rise of Agentic AI architectures offer a new pathway to enhance reliability and security in IoT policy authoring. This paper seeks to fill this gap by presenting AgentHAB, a multi-agent workflow, providing a new dataset for cross-platform research. This experiment will involve a prototype Agentic pipeline, AgentHAB, for Natural language to OpenHAB rule generation with syntax-aware iterative validation, a set of preliminary experiments on syntax validity, robustness to paraphrasing, and the impact of our RAG-based context, a research plan for integrating and evaluating live context validation using MCP [2] to prevent semantic errors and rule conflicts. This paper aims to validate the AgentHAB concept. We pose four questions, where RQs 1-3 are addressed by our preliminary evaluation and RQ4 forms our future research plan.

RQ1: Does a syntax-aware, iterative LLM pipeline (AgentHAB) generate syntactically valid OpenHAB rules more often than a single-shot LLM baseline? This research question seeks to evaluate where feedback, refinement, and validation improve the reliability of rule generation. Prior work, such as in [7] and [8], shows that a single-shot LLM prompt does not perform and is prone to hallucination. Here, we examine whether integrating syntax validation after the rule generation step with an iterative repair loop produces a measurable improvement in syntactical correctness.

RQ2: How robust is the pipeline to paraphrased natural-language requests describing the same automation intent? Smart-home users will articulate automations in a non-fixed phrasing through natural language. This question assesses whether the system can handle linguistic diversity by comparing outputs produced from multiple paraphrases of each intent. Robustness is measured in terms of consistency of syntactic validity and semantic correctness of the results across paraphrased prompts.

RQ3: How does documentation-based context (retrieval over context) affect syntax validity and correctness? Because OpenHAB DSL (Domain-Specific Language) has strict grammar rules and many platform-specific constructs, we will examine how retrieval-augmented generation (RAG) influences rule quality. This question is answered through context ablation experiments, removing specific documentation files (e.g., tutorials or examples) or disabling retrieval entirely, to quantify the contribution of contextual grounding.

RQ4: To what extent can a live system context using MCP reduce undefined-item and rule-conflict errors? This question guides our next research phase, where we plan to integrate the MCP-based context validator Agent. By incorporating live item names, types, and existing rules, the system should be able to prevent undefined identifiers and detect potential conflicts before deployment. This RQ positions the future evaluation of runtime-aware generation and conflict detection, even though full MCP integration is still under

2640-7574/26/$31.00 ©2026 IEEE
DOI 10.1109/SANER67736.2026.00027

development.

II. Background and Related Work

OpenHAB [3] is an open-source automation platform for programming and creating smart home systems. Within configurations of these systems, users can create rules that can automate triggers and events for their smart home. But within the complexities of syntax and the complex documentation of new users creating their environments, minor errors or rule conflicts can cause failures and security issues and can be difficult to diagnose. Misconfiguration can lead to unwanted, unsafe behaviour, especially for new users, inadvertently creating rule conflicts.

Our work is positioned within the intersection of 3 main domains: Code generation by LLMs, IoT Security Policy Verification, and Multi-Agent AI Systems Development.

A. IoT Policy Authoring & Security Challenges: The rapid development and adoption of smart devices across many homes has created complex configurations across platforms and ecosystems. As stated in [4], with interacting applications, there could be race conditions where, for example, we have the AC turned on when it reaches a certain temperature to cool the room down, but the AC turns off when the room temperature reaches a temperature that is cool enough, the AC will automatically turn on and off unnecessarily. This and other security and safety issues pose a challenge in policy authoring to ensure that they are both user-friendly and capable of preventing conflicts.

B. Cross-App Conflict Detection and Policy Authoring Solutions: As stated earlier, conflict between policies can cause unwanted behaviour and security threats amongst IoT devices. HOMEGUARD and IoTGuard [1] were presented, which include categorizing CAI (Cross App Interference) threats and detecting them using symbolic execution and Satisfiability Modulo Theory (SMT). Although apps can exist independently, this tool identified over 660 CAI instances, indicating that threats persist even after app installation and configuration.

Security and policy enforcement come with a great challenge, as conventional systems are deprived of automated rule generation and conflict detection. As introduced by [5], policy creation for the general consumer of smart home technologies is mainly for a non-tech background, so the tedious task of learning a tool's policy language becomes a bottleneck in understanding their own smart home ecosystem. This is where the tool iConPAL [5] comes in, which uses a Large Language Model (LLM) to make it easier for users to translate natural language descriptions from user input to a formal policy language. By giving the LLM the context related to the policy language through prompt engineering, over 90 percent of policies were successfully translated and semantically correct.

Other recent advancements include platforms like AutoIoT [6], which leverages LLMs and formal verification to automatically generate conflict-free rules. AutoIoT [6] separates logic and syntax of code generation, which allows for LLMs to create formal verification code for languages that LLMs are not trained on. These approaches highlight the critical importance of making policy authoring more accessible and secure for the requirements of IoT systems. Our work seeks to fill a different gap in pre-deployment prevention. We aim to stop an invalid or conflicting rule from ever being created, integrating validation directly into the authoring stage. This aligns with the broader push for secure and reliable IoT systems through formal methods. However, these approaches are limited if they fail after deployment or require deep technical expertise to configure smart home applications and interpret system behaviour.

C. LLM usage for IoT Policy Authoring and Code Generation: The usage of LLMs to generate code output from Natural language text input is a known and promising field. iConPAL demonstrates the feasibility of natural language to policy output translation [5]. Our work builds on and extends these concepts by addressing limitations by (1) focusing on the OpenHAB environment and the complexities of creating DSL rules, (2) adding a programmatic validation and iterative refinement loop rather than just pure single-shot code generation, and (3) planning for future deployment and live-context-awareness.

Traditional prompt engineering techniques have been transformed in the industry over the years, where task-based workflows are assigned to an LLM. Prompt engineering emerged as effectively harnessing LLMs towards providing a specialized output [6]. Using carefully structured prompts and instructions, alongside relevant context, such as input and output formatting, prompt engineering guides LLM's behaviour for providing quality, relevant outputs.

LLMs such as Codex and the family of GPT models have been shown to generate syntactically correct code [7]. Reliability and performance have been significantly shown to improve with RAG and feedback loops [8]. Without these mechanisms, LLMs can hallucinate output and ignore the original requested constraints.

D. Advances in Multi-Agent Systems in IoT: Recent work from [9] proposed an IoT-driven heartbeat monitoring system, which demonstrates how multi-agent approaches can increase reliance by active monitoring and fault detection. There is also work from [10] which applied Agent AI to intrusion detection in IoT environments. Their framework combined ML algorithms with AI to classify, explain and enhance trust in Intrusion Detection models. These previous works highlight the importance of LLMs beyond policy translation, but in IoT defence mechanisms.

The evolution and work on LLMs from earlier models have allowed for more capabilities. Compared to older models, newer models such as GPT-5 [11], Claude 4.0 Sonnet [12], and Mistral [13] demonstrate greater reasonable abilities and reduced error rates. These advancements create opportunities for our multi-agent framework, where the first agent generates policies and the other validates against conflict user grammar, examples, syntax, and tutorials.

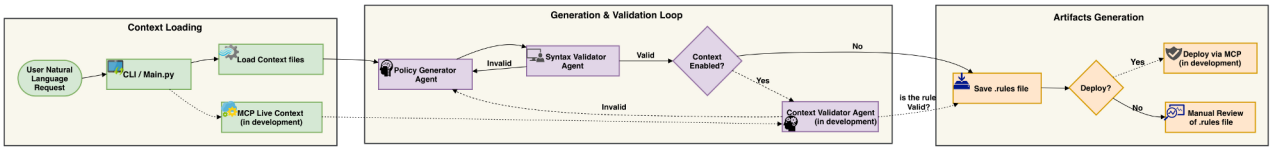

Fig. 1. OpenHAB Agents Architecture diagram: Agent loop with retrieval, generation, validation, and repair.

Our prototyped pipeline architecture is inspired by "agentic" principles, where distinct agents collaborate to solve a complex problem. Using a "Generator" and "Validator" agent is a common pattern for improving LLM reliability, as seen in IConPal [5]. This multi-agent approach has been proposed for scalable IoT security, smart home management, and other complex engineering tasks. Frameworks like IoT-LLM [14] specifically use RAG to enhance an LLM's reasoning for real-world IoT tasks, validating our RAG-based approach. We are applying these agentic principles to the specific software engineering task of DSL rule generation.

Previous works, such as HOMEGUARD and iConPAL, reveal a gap in IoT conflict prevention. HOMEGUARD detects conflicts post-deployment of IoT apps, and IConPAL lowers the barrier to authoring policies, but there remains no system that is unified in both approaches. These implementations from [9] and [10] provide evidence that Agentic frameworks bridge this gap, and further prove motivation for our proposed solution.

III. System Overview: AgentHAB

This agentic system floors a multi stage workflow involving retrieval, generation, validation and optional live context check (in active development). The main components to highlight are (see Figure 1):

1) RAG Context and Retrieval Layer: This workflow uses a BM25 retriever, as seen in `context_loader.py` over the curated context folder corpus. The files in the corpus contain OpenHAB DSL grammar and syntax rules, examples, and a tutorial from the official documentation of OpenHAB [15].

2) Rule Generation Agent `agents/policy_generator.py`: The LLM (in this case, GPT-4o-mini) is prompted with the user's natural language input and the retrieved context from (1). It is instructed in the structured prompt to generate the OpenHAB DSL rule. This Agent is also looped back to if there is feedback from previous "failed" validation attempts from the validator agent, as seen in (3).

3) Validator Agent `agents/validator_agent.py`: The output from the generation Agent gets passed to the validator agent and checks for syntax using a custom parser in `rule_parser.py`. If there is an error detected, the error is passed back to the generator in (2), which enables an iterative refinement in LLM code generation.

4) Iterative Generation and Validation Loop: If the rule is considered "valid", it gets saved in a `.rules` file ready for deployment (which is a part of our next steps in research). If the rule is considered "invalid", the error and the code are sent back to the Generator Agent with new instructions to fix the error. This loop repeats until the retry value is reached, which can be set (defaulted to 3).

TABLE I
Syntax Validity vs Baseline

Configuration	Syntax Validity (By LLM)	Avg. Iterations
Baseline (Single-Shot)	55% (11/20)	1.0
Pipeline (Full)	90% (18/20)	1.45

5) Design Extension: MCP-Based Live Context: The current prototype validates syntax based on the context given. The next phase of our research (see section VI) involves creating or using an external OpenHAB MCP server to connect to our agents' flow. This server uses the OpenHAB REST API to maintain a live check of the "Items", "Things" and "Rules" in the OpenHAB environment [3] through the in-development Context Validator Agent. This would enable full context-aware validation by detecting anything undefined, detecting conflicts with generated and deployed rules and evaluating safety conditions (e.g., "Item LivingRoom_Light does not exist"). This component is designed but not yet fully implemented in the evaluated prototype.

IV. Preliminary Evaluation and Results

Experimental Setup: For our experiments, we used GPT-4o-mini with a temperature of 1.5 in our Agentic workflow to generate rules. We created an initial 20 natural language rules in a dataset with a parallel dataset of a paraphrased version (which is more or less a structured version of the initial prompt, keeping the details) of the initial dataset (some rules and JSON structure taken from the iConPAL dataset in the repository [16]). Each result generated from the scripts is a JSON file, which is then analyzed. We look at the number of interactions and syntax validity to demonstrate the effectiveness of our Agentic pipeline on OpenHAB rule generation.

Syntax Validity vs Baseline: We establish a baseline by just using the Validator Agent with no context to produce a candidate rule in one shot, prompting. We run the initial natural language dataset on the single-shot method and also on our pipeline to compare syntax validity (decided by the LLM) and the average iterations to generate each rule. The Single Shot baseline produced "Syntactically Valid" code for 55% of the time. Our full pipeline achieved a 90% validity score. The 1.45 iterations represent the number of times the

TABLE II
ROBUSTNESS TO PARAPHRASING

Dataset	Syntax Validity (By LLM)	Avg. Iterations
Initial	90% (18/20)	1.45
Paraphrased	75.0% (15/20)	1.7

TABLE III
EFFECT OF CONTEXT REMOVAL

Configuration	Syntax Validity (LLM)	Syntax Validity (Human)	Avg. Iter.
Only Syntax Context + Full	75.0% (15/20)	70% (14/20)	1.7
All Context + Full Agentic Pipeline	90% (18/20)	90% (18/20)	1.45

magnetic trial loop was triggered to correct the initial LLM attempts to achieve the result. Our Muti-Agent AgentHAB System demonstrates that iterative validation is critical.

Robustness to Paraphrasing: We ran our multi-agent pipeline with the initial dataset and then the paraphrased dataset to see how the output is impacted by rewording the text from a less conditional state to a more conversational tone. The initial dataset, as seen previously, has been categorized at 90% valid output with 1.45 iterations. However, the paraphrased dataset sees a 75% valid output with 1.7 iterations, seeing a 15% decrease with 0.25 more iterations. While the overall performance decreased slightly, it can still suggest that the exact phrasing has minimal impact and is mostly robust to variation.

Effect of Context Removal: Context retrieval is essential in our multi-agent system that guides the LLM to create correct and sound OpenHAB rules. Here, we will only consider the syntax markdown file as the only context, while removing the examples, grammar and tutorials markdown files, and compare the results of the workflow to the full context runs we have seen previously. Since the configuration of the context only contained the syntax, we decided to add an extra element of human verification to see if the LLM's results were valid. With an average of 70 to 75 percent Syntax Validity and 0.25 increase in interactions, this demonstrates that a full context with RAG containing all grammar, syntax, tutorials and examples is crucial for generating correct and specialized OpenHAB rules.

V. RESEARCH PLAN AND FUTURE WORK

Our preliminary evaluation shows that AgentHAB is a promising approach for producing syntactically valid OpenHAB rules. The early results form the foundation for our primary research goal: ensuring semantic correctness and conflict-free safety for generated automations. This section describes the planned work centred on RQ4 (live system context via MCP), and outlines experiments, dataset expansion, model and system improvements, and human-in-the-loop evaluation needed to move from syntax to safety.

Full MCP Integration: We will complete a production-grade integration of an OpenHAB MCP component that exposes a REST API for the ValidatorAgent. The MCP server will provide real-time access to a live OpenHAB instance's Items (names, types, metadata, states), Things and channels (device mapping) and existing rules (text and parsed structure).

The ValidatorAgent will be extended to query this API during validation. For any candidate rule, the pipeline will (1) check that referenced items exist and match expected types; (2) flag undefined or ambiguous identifiers; and (3) return structured corrective feedback (e.g., "Item LivingRoom_Light not found — suggestions: Living_Room_Lamp, LR_Light").

Semantic Validation and Conflict Detection: We will implement staged conflict analysis integrated with MCP:

- **Static Conflict Detection (SCD):** Use the live ruleset to detect common structural conflicts before deployment:
  - *Redundancy:* identical or overlapping conditions already embodied in an existing rule.
  - *Direct contradiction:* opposite commands issued for the same item under identical triggers.
  - *Trigger overlap and race conditions:* rules that produce oscillation or unintended toggling.
- **Dynamic / Runtime Checks:** Design lightweight simulation or runtime monitors to detect potential behavioural conflicts (e.g., oscillation due to action sequences). If feasible, lightweight sandboxing (dry-run executions on a virtual instance) will be explored.

For each detected issue, the ValidatorAgent will produce actionable guidance (e.g., suggested renames, condition refinements, or confirmation prompts) that feeds back into the generation loop.

Experimental Plan with Live Context: In response to RQ4, We will re-run and extend our experiments using the fully integrated MCP-enabled pipeline:

- **Dataset:** expand to 50–100 base each, covering lighting, HVAC (Heating, Ventilation, and Air Conditioning), security, notifications, and composite automations. Include real user utterances collected via a small pilot study.
- **Conditions:** (a) Baseline single-shot LLM (no retrieval, no MCP), (b) AgentHAB without MCP (current), (c) AgentHAB with MCP static checks, (d) AgentHAB + MCP + SCD + optional dynamic checks.
- **Metrics:** undefined-item error rate, number of potential conflicts flagged pre-deployment, syntax validity, semantic correctness (human-rated), mean iterations to a valid rule, and deployment success rate (for rules that are actually deployed to a sandbox instance).
- **Hypotheses:** MCP context will materially reduce undefined-item errors and catch a meaningful portion of pre-deployment conflicts, improving deployment safety.

Datasets & Broader Rule Coverage: To improve external validity, we will curate and publish an expanded corpus:

- Ensure representation across device types, naming conventions, regional/locale patterns, and composite workflows.
- Provide train/test splits and metadata for reproducibility.
- Release evaluation prompt sets and expected canonical rules where possible.

This broader dataset enables better evaluation of generalization and robustness across real-world naming conventions and DSL usage.

Diverse LLMs, Model Tuning, and Output Controls: We will evaluate multiple LLMs with varying reasoning and code generation capabilities (e.g., Code Llama, Claude Models, newer OpenAI family models):

- **Model selection:** compare at least three models spanning smaller cost-efficient models and larger reasoning-capable models.
- **Tuning controls:** systematically test settings such as temperature, token output limits, and repetition penalties to reduce hallucination and limit token-budget exposure. We will empirically determine optimal temperature ranges (e.g., compare 0.0, 0.2, 0.5, 1.0) for DSL generation.
- **Cost management:** implement cost caps and budgeted inference (early stop, response length limits), and measure cost-to-quality tradeoffs (quality per $).
- **Prompt caching mitigation:** to avoid artificially improved iteration counts due to prompt caching or LLM exposure to repeated prompts, experiments will randomize prompt order, use cold starts where possible, and track cache artifacts. We will log model latency and iteration counts to detect caching effects.

Human-in-the-Loop & UI Improvements: We will replace the CLI-centric workflow with a prototype human-in-the-loop UI to support interactive review and safe deployment:

- **Reviewer UI:** display generated rule, inline validator diagnostics, suggested fixes, and quick-edit capabilities.
- **Approval flow:** reviewers can accept, edit, or reject rules; edits will be fed back to agents to improve suggestions.
- **Usability study:** conduct a comparative user study with participants writing rules manually vs using AgentHAB + UI; measure task completion time, error rates, and user satisfaction.

This interactive workflow supports both safety (human oversight) and data collection for improving models.

Instrumentation & Agent Logging: To facilitate debugging, reproducibility, and error analysis, we will collect rich logs for every run:

- Prompt/response history for each agent (anonymized), retrieved context chunks, validator JSON reports, iteration counts, and MCP queries/responses.
- Aggregated statistics for model latency, token usage, and cost.
- Structured failure categories (syntax, undefined item, conflict type).

These logs will support ablation studies, error classification analysis, and future supervised learning to improve prompt construction.

Anticipated Outcomes & Impact: We expect MCP integration to substantially reduce undefined-item errors and proactively flag common static conflicts, improving deployment safety. The expanded dataset and multi-LLM evaluation will help generalize findings, while the interactive UI and agent logs will support practical adoption and iterative research improvements. Together, these efforts move AgentHAB from syntactic assistance to a practical, safety-aware authoring tool for evolving IoT systems.

VI. Threats to Validity

Internal Validity: Our preliminary correctness ratings (Section V) were conducted solely by the authors, introducing subjectivity and potential confirmation bias. In addition, the evaluation uses a relatively small prompt set (20 paraphrases), which may not adequately represent the diversity of real-world smart-home requests. Another internal threat concerns potential prompt caching effects in the LLM: if the model has recently seen similar instructions during iterative refinement, it may produce valid rules more quickly in later runs, artificially inflating the apparent effectiveness of the pipeline.

External Validity: The current evaluation uses a single LLM backend (GPT-4o-mini) and targets one smart-home automation platform (OpenHAB). Results may not generalize to more expressive or less structured DSLs, alternative LLMs, or other home-automation ecosystems such as SmartThings or Home Assistant. Broader experiments are required to assess generalizability.

Construct Validity: We rely primarily on Syntax Validity and light-touch semantic scoring as proxies for rule quality. A syntactically correct rule can still encode unsafe behaviours, conflicting triggers, or unintended automation logic. This limitation motivates our planned extension in Section VI, where we introduce live-context validation and conflict analysis as a means to capture deeper correctness criteria.

Implementation Status: The live system context validation targeted by RQ4 is part of our research plan and not included in the results presented in Section V. The MCP-based context validator exists in prototype form but is not yet integrated into the main evaluation loop. This restricts our current findings to syntax-level guarantees and represents an acknowledged limitation of the preliminary results.

VII. Conclusion

This paper presented AgentHAB, working toward reliable, natural-language-driven generation of openHAB automations using a multi-agent LLM pipeline. Preliminary findings show substantial improvements in syntactic validity and robustness compared to single-shot prompting, and highlight the importance of documentation-driven retrieval. Future work focuses on integrating live system context through MCP, enabling runtime-aware validation and conflict detection. We aim for this work to serve as a foundation for safe, easy-to-use, reproducible, and accessible automation authoring tools in evolving IoT software ecosystems.

Data Availability

The code, datasets, and results are available at `https://github.com/cresset-lab/AgentHAB`.

ACKNOWLEDGMENT

We acknowledge the use of Cursor Agent (utilizing Claude-4.5-Sonnet and GPT-5.1) to assist in brainstorming and conceptualizing the logic for the `policy_generator.py` and `validator_agent.py` agents, as well as the batch generation scripts within the `agent_pipeline` and `baseline` folders. All final content and code implementations were authored and verified by the researchers.

REFERENCES

[1] H. Chi, Q. Zeng, X. Du, and J. Yu, "Cross-App Interference Threats in Smart Homes: Categorization, Detection and Handling," in 2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), IEEE, 2020, pp. 411–423. doi: 10.1109/DSN48063.2020.00056

[2] "MCP", https://modelcontextprotocol.io/docs/getting-started/intro

[3] "OpenHAB Documentation", https://www.openhab.org/docs/

[4] L. Abuserrieh and M. H. Alalfi, "A Survey on Verification of Security and Safety in IoT Systems," IEEE access, vol. 12, pp. 138627–138645, 2024, doi: 10.1109/ACCESS.2024.3413071

[5] M. Alam, S. Zhang, E. Rodriguez, A. Nafis, and E. Hoque, "iConPAL: LLM-guided Policy Authoring Assistant for Configuring IoT Defenses," in 2024 IEEE Secure Development Conference (SecDev), IEEE, 2024, pp. 76–92. doi: 10.1109/SecDev61143.2024.00014

[6] Y. Cheng, M. Xu, Y. Zhang, K. Li, R. Wang, and L. Yang, "AutoIoT: Automated IoT Platform Using Large Language Models," IEEE internet of things journal, vol. 12, no. 10, pp. 13644–13656, 2025, doi: 10.1109/JIOT.2024.3523907

[7] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching Large Language Models to Self-Debug," 2023, doi: 10.48550/arxiv.2304.05128

[8] Y. Wang, S. Guo, and C. W. Tan, "From Code Generation to Software Testing: AI Copilot with Context-Based RAG," 2025, doi: 10.48550/arxiv.2504.01866

[9] Vishwanadham Mandala, "Revolutionizing Engine Manufacturing: IoT-Driven Heartbeat Monitoring with Agentic AI Innovations," Journal of information systems engineering & Management, vol. 10, no. 22s, pp. 794–803, 2025, doi: 10.52783/jisem.v10i22s.3621

[10] Chandrani Mukherjee,"Use of Agentic AI with LLM and Prompt Engineering and State-of-the-Art Machine Learning Algorithm to detect the patterns in IOT Device Network Intrusion Attacks", doi: 10.36227/techrxiv.174702071.17901547/v2

[11] "GPT-5", https://openai.com/gpt-5/.

[12] "Claude-4.0-Sonnet", https://www.anthropic.com/news/claude-4.

[13] "Mistral", https://mistral.ai/.

[14] T. An, Y. Zhou, H. Zou, and J. Yang, "IoT-LLM: Enhancing Real-World IoT Task Reasoning with Large Language Models," 2024, doi: 10.48550/arxiv.2410.02429

[15] "OpenHAB Textual Rules", https://www.openhab.org/docs/configuration/rules-dsl.html#defining-rules

[16] "IConPAL", https://github.com/syne-lab/iconpal